Pipelined processors

M. Sonza Reorda

Politecnico di Torino
Dip. Automatica e Informatica



Introduction

To improve the performance of a processor, you can work in two directions:

- **increasing the speed of the components**
- **changing the architecture.**

The use of pipelined architectures follows the second road.

RISC and superscalar processors

Traditional processors require a certain number of clock cycles to execute each instruction.

Thanks to the introduction of pipelines, in every clock cycle

- **RISC processors can often complete one instruction**
- **superscalar processors can often complete more than one instruction.**

CPI

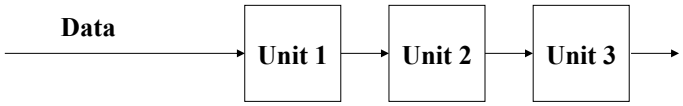
To measure the efficiency of a processor we use sometimes the parameter CPI (*Clocks per Instruction*):

- **For CISC processors ⇒ CPI > 1**
- **For RISC processors ⇒ CPI ~ 1**
- **For superscalar processors ⇒ CPI < 1.**

Pipeline

The pipeline is the electronic equivalent of the assembly line.

It allows the processor to perform multiple operations on the same data stream in parallel.



Ideal pipeline

If

- **all data go through the same stages**
- **all stages always require the same time to perform their processing**

then the use of a *n*-stage pipeline can lead to a *n* times improvement in performance.

Pipeline in processors

Execution of each statement can be divided into several phases, such as:

- ***Fetch*: it fetches one instruction from memory**
- ***Decode*: it decodes one instruction and read its source operands**
- ***Operate*: it executes the required operation**
- ***Write*: it writes the result.**

Each processor can subdivide instruction execution in a different number of phases:

- **ARM processors: 3 phases**
- **Intel Netburst Processors (Pentium 4): 31 Phases**

7

7

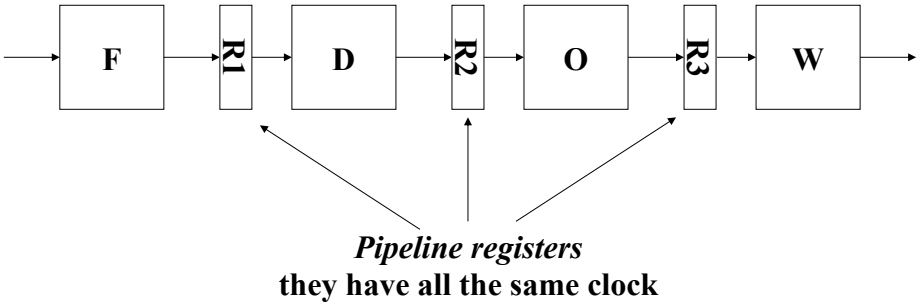
Intel Microarchitecture Codename	Pipeline stages
P5 (Pentium)	5
P6 (Pentium 3)	10
P6 (Pentium Pro)	14
NetBurst (Willamette)	20
NetBurst (Northwood)	20
NetBurst (Prescott)	31
NetBurst (Cedar Mill)	31
Core (Core2)	14
Bonnell (Atom)	16
Sandy Bridge (1st generation 13, 15 and i7)	14
Silvermont (Atom, Celeron and Pentium for SoC)	14-17
Haswell (4th generation 13, 15 and i7)	14
Skylake (Xeon)	14
Kabylake	14

8

8

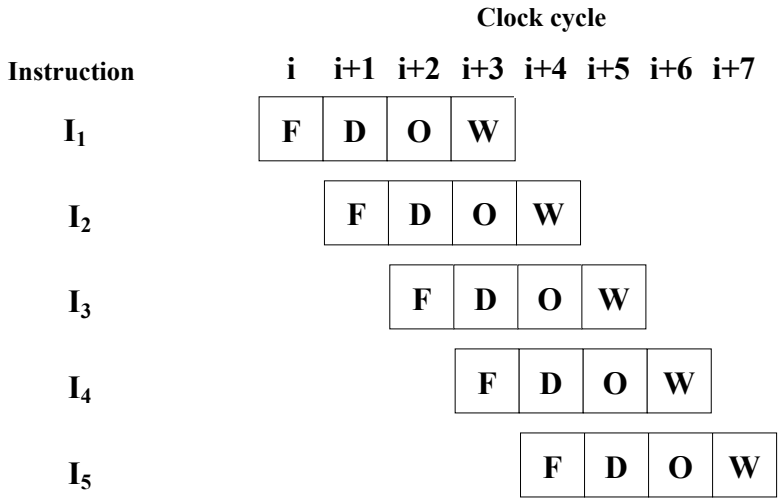
Architecture

Pipeline stage
In each clock cycle it executes one phase on one instruction

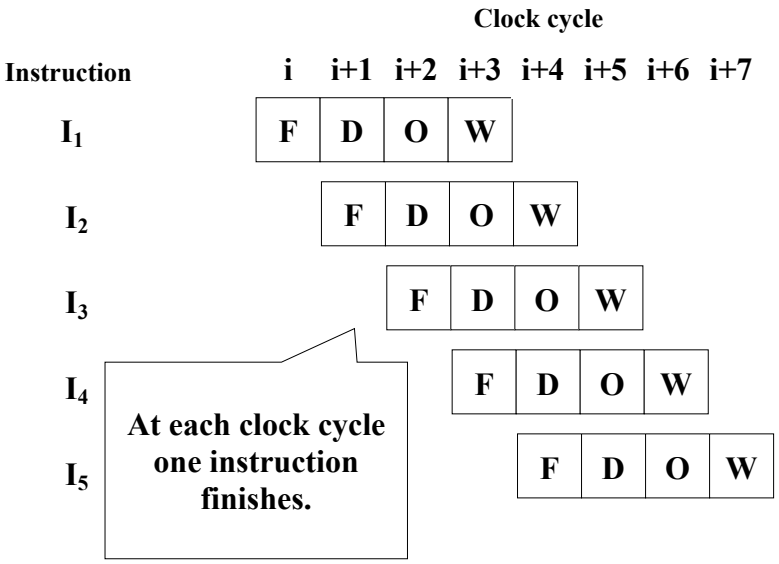


Pipeline registers
they have all the same clock

Example



Example



11

Stall

In a real processor, sometimes a stage can not complete its task in a clock cycle.

Then

- the other upstream stages must stop (that is, stall), as the registers they are to write are not free
- downstream stages can continue to work, but only to work on the instructions already processed by the critical stage.

In this way the performance of the processor drops compared to the ideal ones.

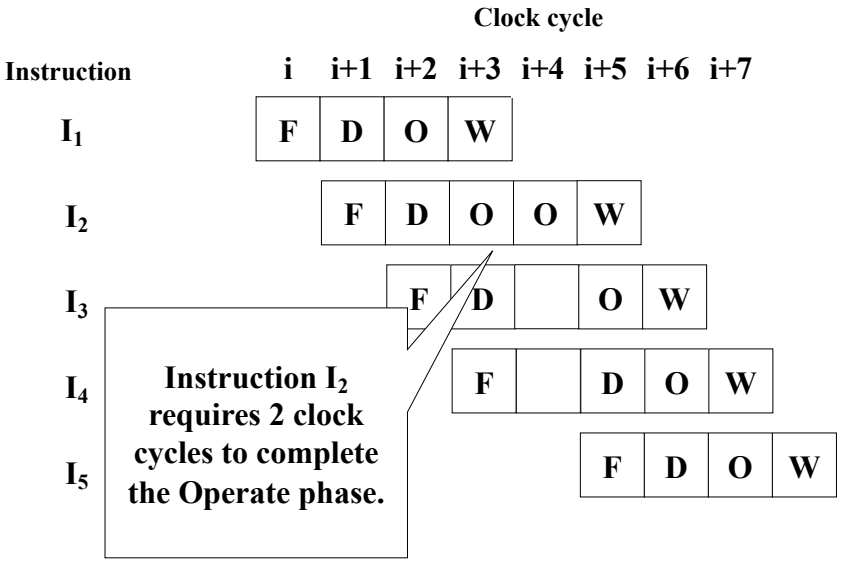
12

Stall causes

- Slow memory accesses (in phases F, D and W)
- Slow execution of internal operations (phase O).

13

Example



14

Caches

Caches have a key importance in pipelined processors, as they allow the processor to access data and instructions in just one clock cycle.

Bubbles

After a stadium has stalled, an inactivity situation (called *bubble*) is created within the pipeline, that will be only gradually eliminated.

Instruction queue

When the fetch unit does not find an instruction in cache (miss), it stalls the entire pipeline.

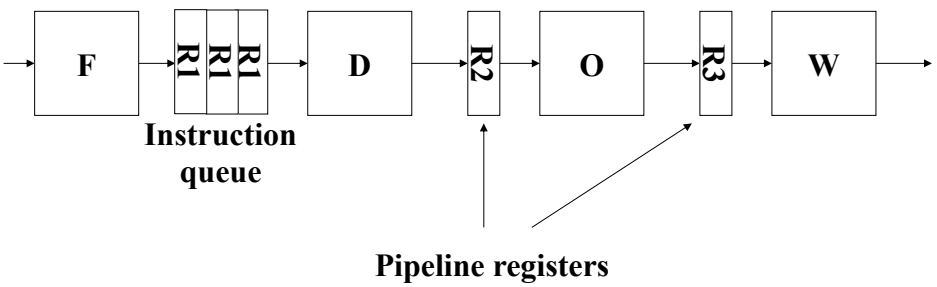
To avoid this situation, the buffer between the fetch and decoding units is replaced by a queue, called the *instruction queue*.

The fetch unit tries to keep this queue full, loading even more than one instruction per clock cycle.

This will compensate for any delays due to cache misses.

17

Architecture



18

Data dependencies

The presence of a pipeline makes parallel a process (the one instruction execution) that is originally sequential.

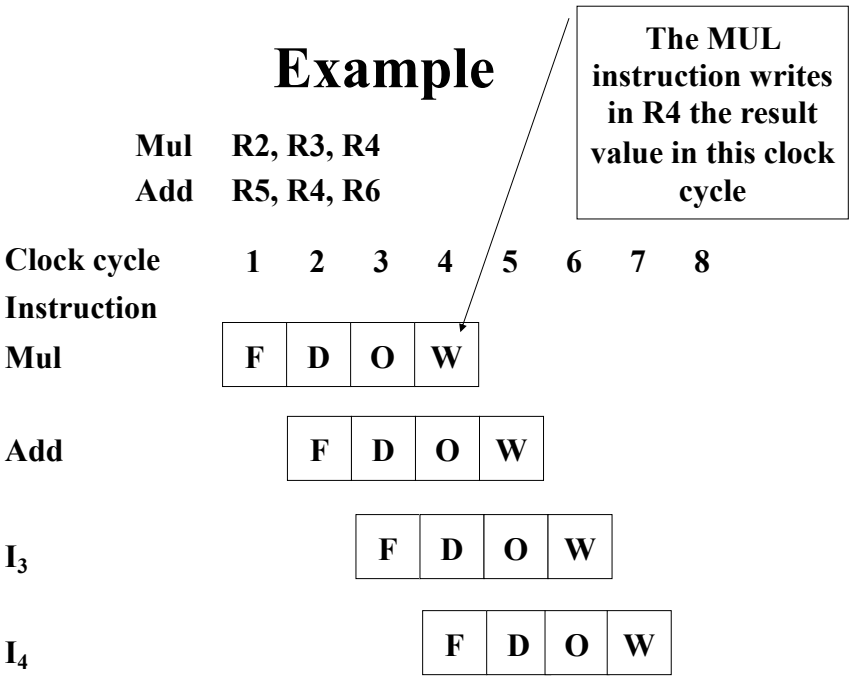
For the result to be the same as in the original process, there must be no data dependency.

We have a data dependency when one of the source operands of an instruction are produced by a previous instruction.

19

Example

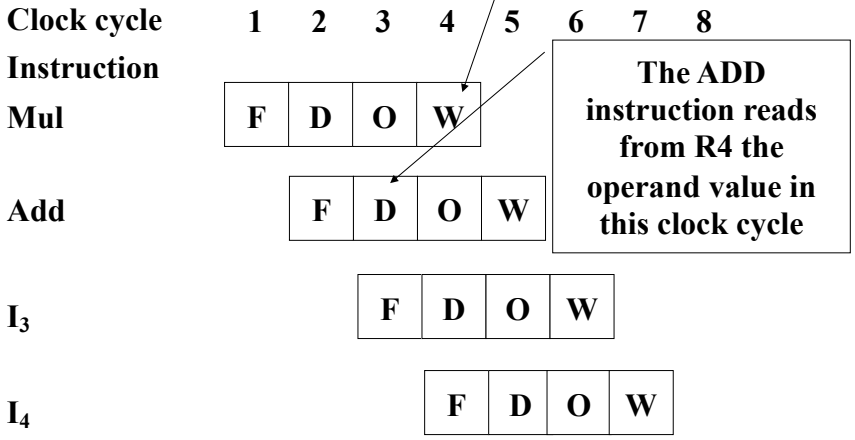
Mul R2, R3, R4
Add R5, R4, R6



20

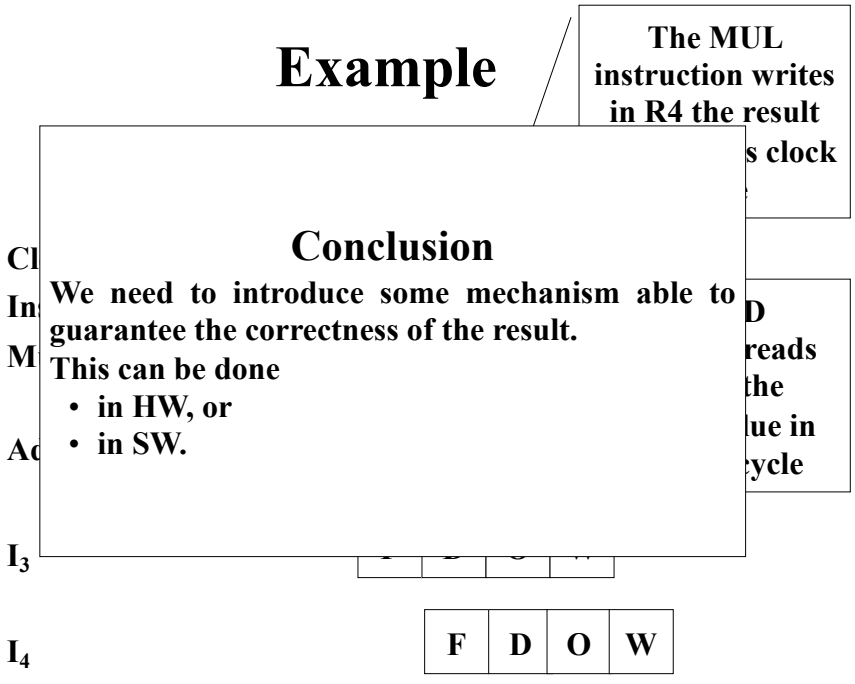
Example

Mul R2, R3, R4
Add R5, R4, R6



21

Example



Conclusion

We need to introduce some mechanism able to guarantee the correctness of the result.
This can be done

- in HW, or
- in SW.

22

HW solution

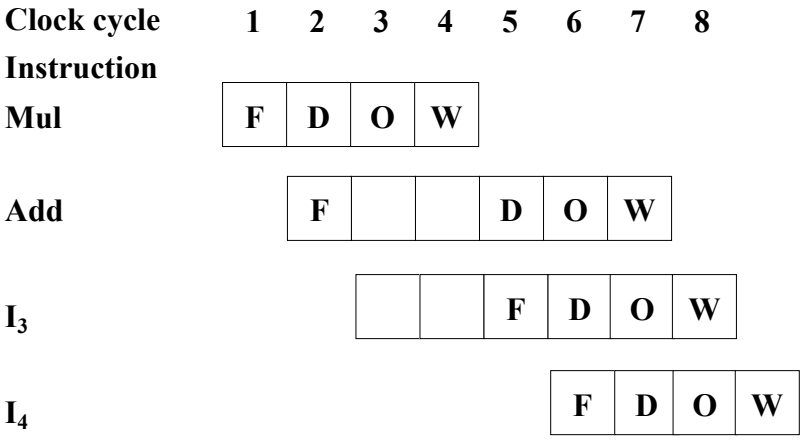
The pipeline HW is able to

- **recognize data dependency situations**
- **block specific pipeline stages (introducing stalls) to ensure the correctness of the result.**

23

Example

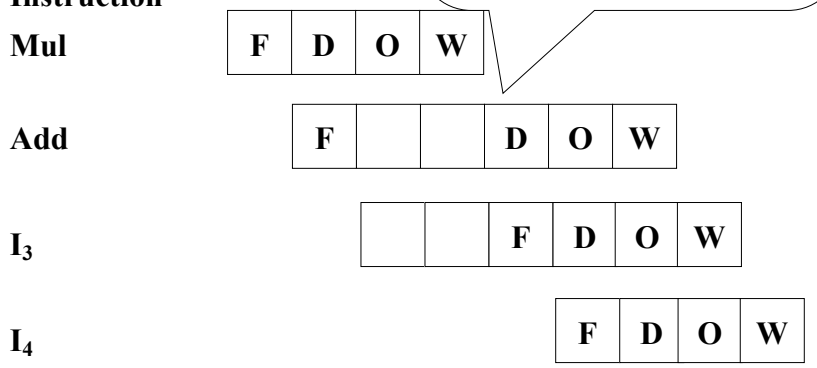
Mul R2, R3, R4
Add R5, R4, R6



24

Exam

Mul R2, R3, R4
Add R5, R4, R6



The Decode phase cannot be executed until the Write phase of the previous instruction is executed, which produces the updated version of R4.

25

SW solution

The compiler introduces some NOP instructions, which allow the pipeline to produce a correct result.

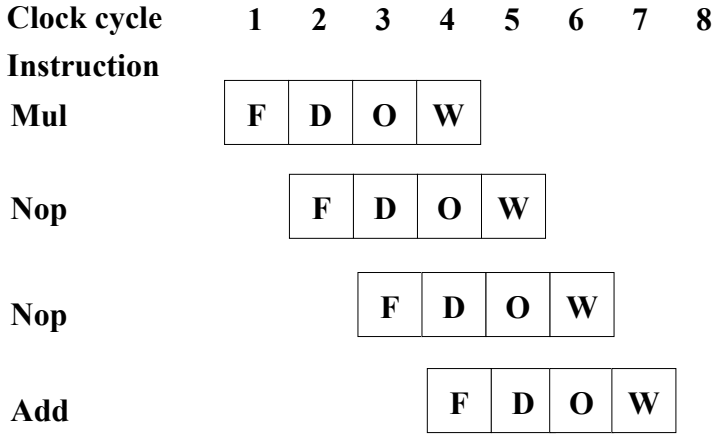
Example

Original code		Modified code	
I_i		I_i	
Mul	R2, R3, R4	Mul	R2, R3, R4
Add	R5, R4, R6	NOP	
I_j		NOP	
I_k		Add	R5, R4, R6
…		I_j	

26

Example

```
Mul     R2, R3, R4
Nop
Nop
Add     R5, R4, R6
```

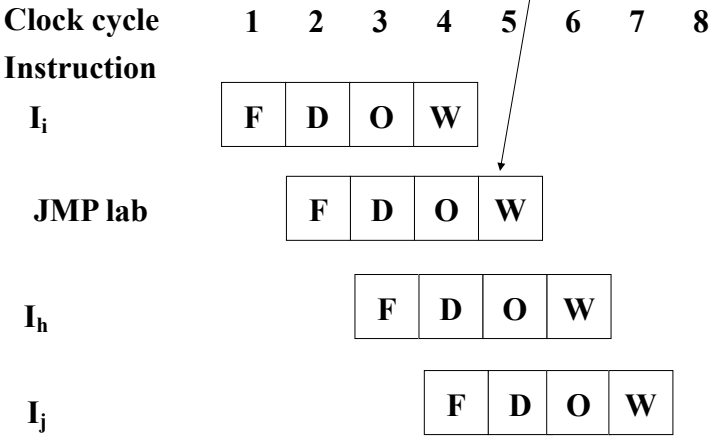


Jump instructions

Jump instructions are potentially very damaging to the performance of the pipeline, as they interrupt the normal sequential execution flow.

Example

I_i
JMP lab
I_h
I_j
lab: I_k

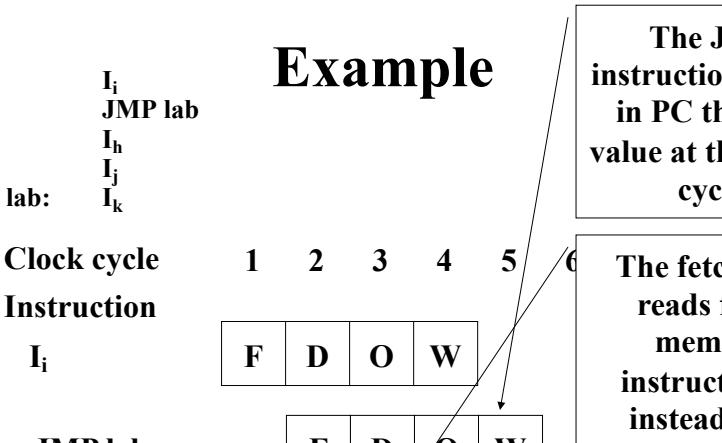


The JMP instruction writes in PC the new value at this clock cycle

29

Example

I_i
JMP lab
I_h
I_j
lab: I_k

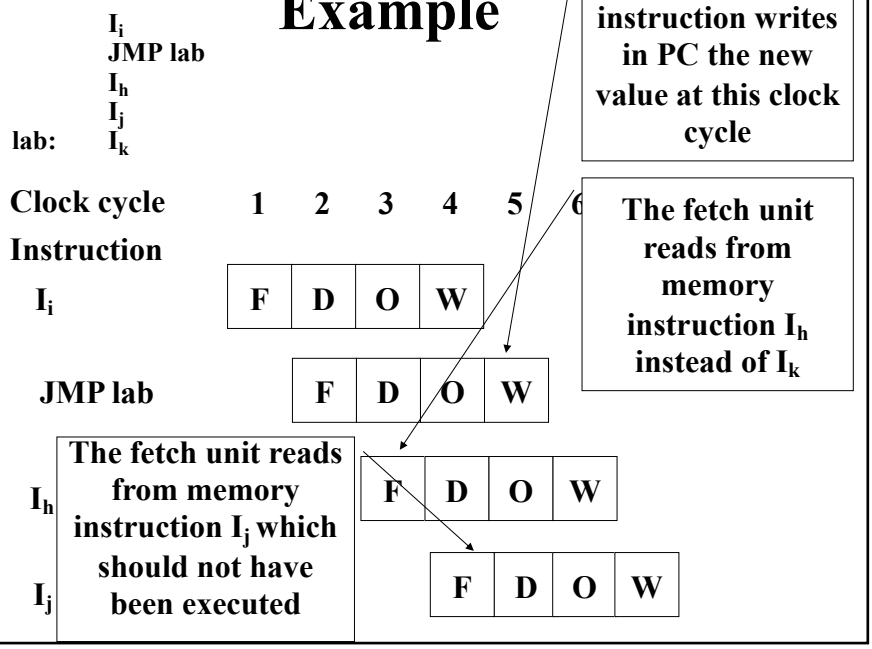


The JMP instruction writes in PC the new value at this clock cycle

The fetch unit reads from memory instruction I_h instead of I_k

30

Example



Branch delay slot

The *branch delay slot* is the number of clock cycles that follow a jump instruction, in which the processor fetches using a value of the PC that is not necessarily correct.

Solutions

To ensure the accuracy of the results produced by the processor even in the presence of jumps you can again use two groups of solutions:

- **HW techniques**
- **SW techniques.**

33

HW techniques

They can

- **detect jump instructions, and then empty the pipeline from erroneously fetched instructions**

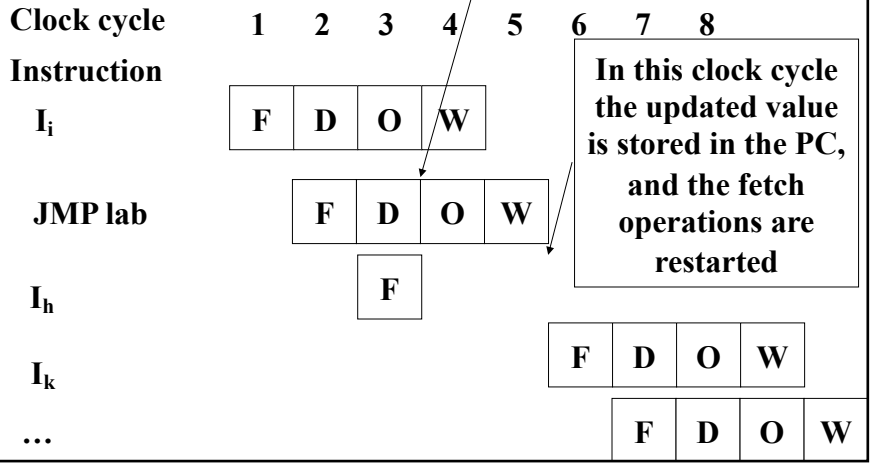
or

- **implementing more sophisticated techniques, depending on whether the jump is unconditional or conditional.**

34

Example

```
        I_i
        JMP lab
        I_h
        I_j
lab:    I_k
```



In this clock cycle the CPU realizes that the current instruction is a jump and stops fetching new instructions

In this clock cycle the updated value is stored in the PC, and the fetch operations are restarted

SW techniques

They are based on

- *Delayed branch*
  introduction of NOP instructions by the compiler (after each jump it introduces in the code as many NOPs as the branch delay slot)

or

- Code optimization.

Delayed branch

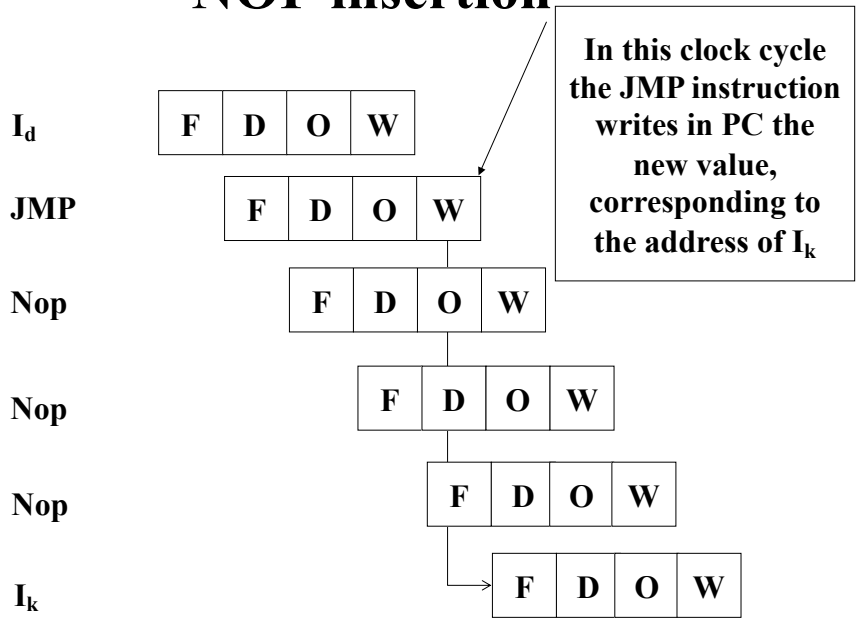
The compiler introduces NOP instructions, which allow the pipeline to produce a correct result.

Example

Original code	modified code
I_a	I_a
I_b	I_b
I_c	I_c
I_d	I_d
JMP lab_1	JMP lab_1
I_j	NOP
Lab_1: I_k	NOP
	NOP
	I_j
	Lab_1: I_k

37

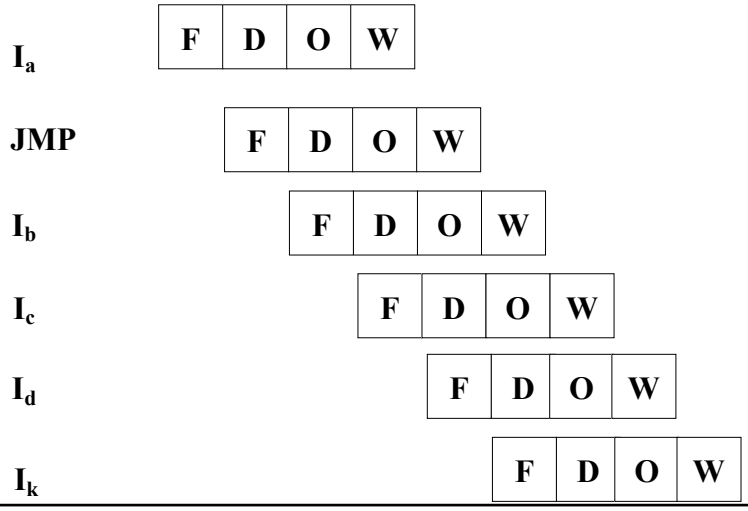
NOP insertion



38

Optimization

The jump instruction is anticipated and executed before a number of instructions (if possible) equal to the branch delay slot, so that the processor does not have to execute any NOP instructions.



Conditional branches

In this case, you must wait for the instruction preceding the jump one to know whether the jump has to be executed or not.

Meanwhile, however, the instruction queue has been filled sequentially.

If the jump is executed, there will be some instructions that should not be executed (as many as the branch delay slot size).

Branch prediction

If a HW technique is used, prediction reduces the penalty imposed by a jump instruction.

Branch prediction consists in providing the fetch unit with a prediction about whether the jump will be done or not.

- **If the forecast is affirmative, the fetch unit will fetch instructions from the jump target destination**
- **if it is negative, the fetch unit will continue to load instructions from the code after the jump instruction.**

Subsequently, we will check if the prediction was correct;

- **If not, the pipeline will be flushed.**

41

RISC processors

From the early 1980s some processors were designed and then manufactured, characterized by

- **High simplicity**
- **Reduced number of instructions (from which the RISC acronym, which stands for Reduced Instruction Set Computer)**
- **High number of registers**
- **Pipelined architecture.**

In the 1990s, several RISC processor families were on the market, including

- **ARM**
- **SPARC (Sun)**
- **MIPS**
- **PowerPC (IBM).**

42

RISC features: 1 instruction = 1 clock cycle

Each instruction corresponds to several operations:

- **fetch operands from registers**
- **ALU activation**
- **saving the result to a register.**

Thanks to the adoption of the pipeline, it is possible to temporarily overlap the execution of several instructions, so that an instruction execution is normally completed at each clock cycle.

In order to make the best use of the pipeline, the instructions must be

- **Simple**
- **Regular (fixed format).**

RISC features: control unit

RISC instructions have the complexity of CISC micro-instructions.

For this reason, the RISC control unit can be implemented using hardwired technique instead of microprogrammed.

RISC features: LOAD & STORE

RISC processors normally have only two instructions involving memory

- **LOAD (memory ⇒ register)**
- **STORE (register ⇒ memory).**

This allows you to concentrate on just 2 instruction the efforts to reduce the negative impact of address calculation and access to memory.

These instructions are the only ones to provide mechanisms for specifying the address to which access to memory (addressing modes).

45

RISC features: addressing modes

RISCs have a limited number of addressing modes, which are only used in the LOAD and STORE instructions.

46

RISC features: instruction format

RISCs have a fixed instruction format: the operating code usually has a fixed length.

Some advantages are achieved in this way:

- **decoding the operating code can occur in parallel with loading the operands from the registers**
- **the control unit is simpler**
- **the fetch phase is more optimized.**

47

Registers

Registers are the form of memory with the smallest access time because:

- **They are on the same CPU chip**
- **They are built with the fastest technology**
- **They are accessible with a simple addressing mechanism.**

You can improve the execution efficiency in 2 ways:

- **increasing the number of registers**
- **optimizing their use.**

The registers replace the stack (for example, for storing the parameters for procedures, local variables, and return address), reducing the number of accesses in the stack (which is in memory).

48

Machine code size

- **Compilers for RISC are more complex than those for CISCs (while producing code consisting of simpler instructions) as they have to optimize code to avoid stalls**
- **The code generated for a RISC has comparable dimensions to those for a CISC as:**
  - **– the number of generated RISC statements is greater, but**
  - **– each instruction occupies fewer bytes**
- **A shorter code is more efficient because:**
  - **– it reduces the number of fetch operations**
  - **– it reduces the number of page faults.**

RISCs: conclusions

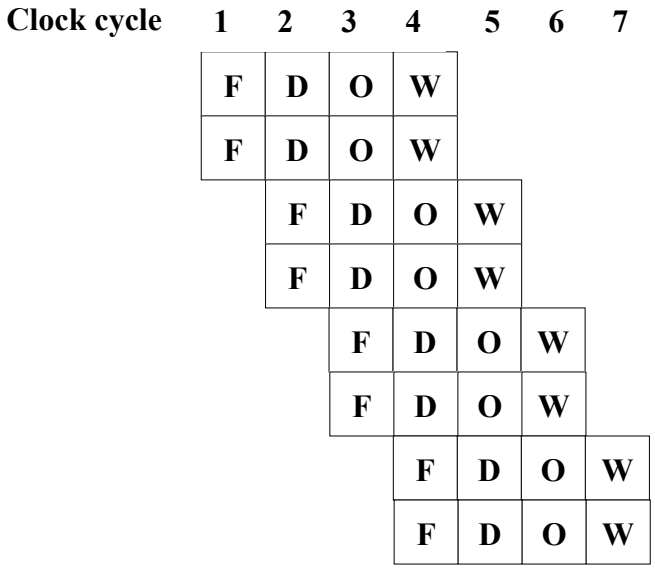
- **CISC instructions, although more complex, are slower, as they require more complicated hardware**
- **Designing an optimized pipeline is simpler for a RISC processor**
- **The availability of a large number of registers can significantly reduce the number of memory accesses**
- **In RISC processors the interrupt latency is lower than for CISCs.**

Superscalar processors

Superscalar processors can complete the execution of more than one instruction for each clock cycle.

Internally, they contain a pipeline in which each stage includes more than one unit.

Ideal behavior

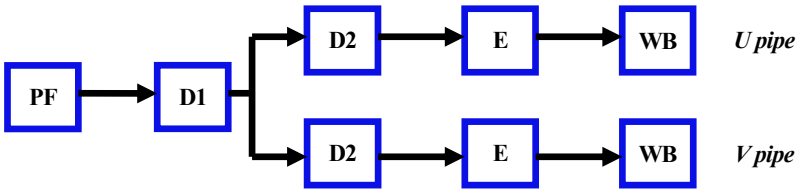


Example: Intel Pentium

It was an Intel processor of the x86 family, produced since 1993

It had 2 pipelines that allowed him to complete more than one instruction per clock cycle:

- **"pipeline U" could execute any instruction**
- **"Pipeline V" was able to execute only the simplest and most common ones.**



53

Number of stages in the pipeline

If it grows, it allows theoretically to increase the performance of the processor.

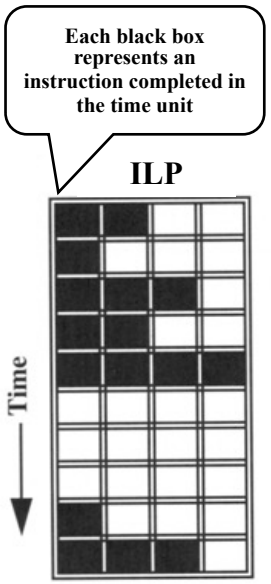
But as the number of stages increases, the likelihood of dependencies increases, and therefore the pipeline has to be stalled.

54

Instruction Level Parallelism

Pipeline processors exploit the fact that the instructions of a program can in many cases be performed in partial overlapping (*Instruction Level Parallelism*, or ILP).

Superscalar processor architectures can best exploit this form of parallelism.



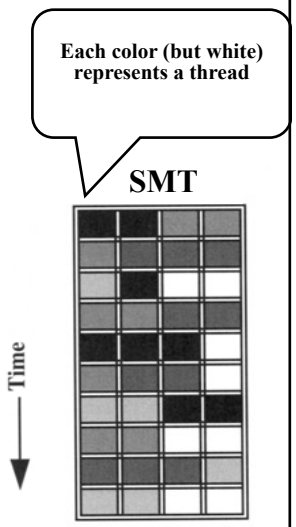
55

Multithread processors

In order to obtain additional performance improvements, it is necessary to identify and exploit different forms of parallelism, such as the thread level (*Thread Level Parallelism*, or TLP).

A thread corresponds to a program portion with its data; a program can be partitioned over multiple threads.

***Simultaneous Multi-Threading* (SMT) processors run multiple threads simultaneously.**



56

Multi-Threading

Multithreading: the CPU splits physical cores into virtual cores (threads) to allow each core to run 2+ "instruction streams at once"

Note: Intel branded this process as hyper-threading

Instruction-level Parallelism
vs.
Thread-level Parallelism

Simultaneous Multi-Threading

Multicore processors

Another recent trend is to integrate in the same device multiple processor cores.

Each processor is relatively simple, and can run multiple threads.

In this way (if you are able to exploit all the cores) you get high performance with low power consumption.

Example: Intel Sandy Bridge

The Intel Sandy Bridge architecture was developed since 2005. The first product based on this architecture was marketed starting in 2011.

The architecture allows to combine up to 8 cores.

Other features:

- L1 Cache: 32 kB data + 32 kB instruction per core
- L2 Cache: 256 kB per core
- Shared L3 Cache (up to 20MB).



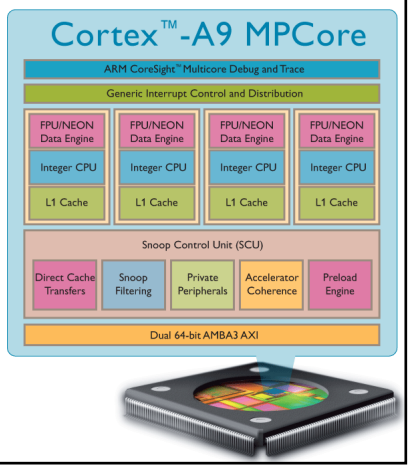
59

Example: ARM Cortex-A9

ARM produces IP processor cores for embedded applications.

Cortex-A9 can be used alone or in multi-core configurations.

It is a solution for applications where high performance and low power consumption is required.



60